

Wiggle Subsequence Explanation

1. What is the problem asking?

We have an array `nums`.

We need to choose some elements **without changing their order** so that the chosen elements form a **wiggle sequence**.

A wiggle sequence means:

- first difference can be positive or negative
- then the next difference must have the **opposite sign**
- then opposite again
- and so on
- **zero difference is not allowed**

For example:

```
[1, 7, 4, 9, 2, 5]
```

```
1 → 7  +
7 → 4  -
4 → 9  +
9 → 2  -
2 → 5  +
```

```
+ - + - +
```

So the answer is 6.

Important: Subsequence ≠ Subarray

We can skip elements.

For:

```
[1, 17, 5, 10, 13, 15, 10, 5, 16, 8]
```

we could select:

```
[1, 17, 10, 13, 10, 16, 8]
```

We skipped some elements, but their original order is preserved.

2. Key observation

The most important thing to notice is:

We don't actually care about the exact values. We mainly care whether the next move is **up** or **down**.

Suppose we have:

```
1 → 7 → 4 → 9
```

The pattern is:

```
UP → DOWN → UP
```

Every time the direction changes, we can extend our wiggle subsequence.

So we want to maximize the number of:

```
UP, DOWN, UP, DOWN...
```

changes.

3. Basic / brute-force idea

A natural first thought is:

Try every possible subsequence and check whether it is a wiggle sequence.

For an array of size n , there are:

```
2^n
```

possible subsequences.

For example, with:

```
[1, 7, 4]
```

we have:

```
[ ]
[1]
[7]
[4]
[1,7]
[1,4]
[7,4]
[1,7,4]
```

Checking all of them becomes extremely expensive.

Complexity

```
O(2^n)
```

Clearly not good.

4. DP idea

We can improve this.

For every index i , we can ask:

up[i]

What is the longest wiggle subsequence ending at i where the **last difference is positive**?

down[i]

What is the longest wiggle subsequence ending at i where the **last difference is negative**?

For example:

```
nums = [1, 7, 4]
```

Initially:

```
up = [1, 1, 1]
down = [1, 1, 1]
```

At 7:

```
1 → 7 = positive
```

Therefore:

```
up[1] = down[0] + 1
       = 1 + 1
       = 2
```

At 4:

```
7 → 4 = negative
```

So:

```
down[2] = up[1] + 1
         = 2 + 1
         = 3
```

Result:

```
3
```

This DP solution is $O(n^2)$ if we check previous elements.

But the problem specifically asks:

Can you solve it in $O(n)$?

Yes.

5. The important greedy intuition

Here's the trick.

Suppose we have:

```
[1, 5, 3]
```

We get:

```
1 → 5 = UP
5 → 3 = DOWN
```

Length = 3.

Now imagine:

```
[1, 5, 6, 3]
```

We have:

```
1 → 5 = UP
5 → 6 = UP
6 → 3 = DOWN
```

The two consecutive UP movements are a problem.

Which one should we keep?

We should keep the **largest possible value before going DOWN**.

Instead of:

```
1 → 5 → 6 → 3
```

we can think:

```
1 → 6 → 3
```

The length hasn't increased from the extra 5.

This leads to the greedy idea:

If the direction stays the same, don't increase the answer. Just update the current endpoint to a better value.

6. Even simpler way to think about it

Maintain two values:

```
up
down
```

up

Longest wiggle subsequence whose last movement is **UP**.

down

Longest wiggle subsequence whose last movement is **DOWN**.

When we see:

```
nums[i] > nums[i-1]
```

we have an upward movement.

Therefore:

```
up = down + 1
```

Why?

Because before making an UP move, the previous move must have been DOWN.

Similarly, if:

```
nums[i] < nums[i-1]
```

we have a DOWN movement:

```
down = up + 1
```

If:

```
nums[i] == nums[i-1]
```

nothing changes because:

```
difference = 0
```

and zero is not a valid wiggle movement.

7. Why comparing only adjacent elements works

This is the part that often feels confusing.

Consider:

```
[1, 2, 3, 4, 5]
```

We see:

```
1 → 2  UP
2 → 3  UP
3 → 4  UP
4 → 5  UP
```

We cannot count all of these because the directions don't alternate.

So:

```
up = 2
```

and it stays 2.

But when the direction changes:

```
[1, 2, 3, 2]
```

we get:

```
UP
UP
DOWN
```

The **DOWN** allows us to extend:

```
down = up + 1
```

giving:

```
3
```

So essentially:

Every time the direction changes, we can add one element to the wiggle subsequence.

8. Optimal O(n) approach

We start with:

```
C++
int up = 1;
int down = 1;
```

A single element is already a wiggle subsequence of length 1.

Then scan the array:

```
for i = 1 → n-1
```

Case 1: `nums[i] > nums[i-1]`

Current movement is UP.

So:

```
C++
up = down + 1;
```

Case 2: `nums[i] < nums[i-1]`

Current movement is DOWN.

So:

```
C++
down = up + 1;
```

Case 3: Equal

Do nothing.

Finally:

```
C++
return max(up, down);
```

9. C++ Code

```
C++
class Solution {
public:
    int wiggleMaxLength(vector<int>& nums) {
        int n = nums.size();

        if (n == 1)
            return 1;

        int up = 1;
        int down = 1;

        for (int i = 1; i < n; i++) {
            if (nums[i] > nums[i - 1]) {
                // Current difference is positive
                up = down + 1;
            }
            else if (nums[i] < nums[i - 1]) {
                // Current difference is negative
                down = up + 1;
            }
            // Equal -> ignore
        }

        return max(up, down);
    }
};
```

10. Code intuition

The entire algorithm basically comes down to these two lines:

```
C++
up = down + 1;
```

and

```
C++
down = up + 1;
```

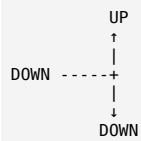
Think of them as:

```
Need UP?
→ Previous must be DOWN
→ Extend DOWN by 1

Need DOWN?
→ Previous must be UP
```

→ Extend UP by 1

So:



The two states continuously build on each other.

11. Complete dry run — Example 1

Input:

[1, 7, 4, 9, 2, 5]

Initial:

up = 1
down = 1

i = 1

nums[1] = 7
nums[0] = 1
7 > 1

So movement is UP.

up = down + 1
= 1 + 1
= 2

Now:

up = 2
down = 1

Subsequence:

1 → 7

i = 2

nums[2] = 4
nums[1] = 7
4 < 7

Movement is DOWN.

down = up + 1
= 2 + 1
= 3

Now:

up = 2
down = 3

Subsequence:

1 → 7 → 4

Pattern:

UP → DOWN

i = 3

9 > 4

Movement is UP.

```
up = down + 1
    = 3 + 1
    = 4
```

Now:

```
up = 4
down = 3
```

Pattern:

```
UP → DOWN → UP
```

Subsequence:

```
1 → 7 → 4 → 9
```

i = 4

```
2 < 9
```

Movement is DOWN.

```
down = up + 1
       = 4 + 1
       = 5
```

Now:

```
up = 4
down = 5
```

Subsequence:

```
1 → 7 → 4 → 9 → 2
```

Pattern:

```
UP → DOWN → UP → DOWN
```

i = 5

```
5 > 2
```

Movement is UP.

```
up = down + 1
    = 5 + 1
    = 6
```

Final:

```
up = 6
down = 5
```

Therefore:

```
C++
max(up, down) = 6
```

Answer

```
6
```

The entire array is already a wiggle sequence:

```
1 → 7 → 4 → 9 → 2 → 5
+   -   +   -   +
```

12. Dry run — Example 2

```
nums = [1,17,5,10,13,15,10,5,16,8]
```

Let's track only **up** and **down**:

Current comparison	Direction	up	down
Start	—	1	1
1 → 17	UP	2	1
17 → 5	DOWN	2	3
5 → 10	UP	4	3
10 → 13	UP	4	3
13 → 15	UP	4	3
15 → 10	DOWN	4	5
10 → 5	DOWN	4	5
5 → 16	UP	6	5
16 → 8	DOWN	6	7

Final:

```
up = 6
down = 7
```

Therefore:

```
answer = 7
```

One valid subsequence is:

```
[1, 17, 10, 13, 10, 16, 8]
```

Its differences are:

```
+16
-7
+3
-3
+6
-8
```

So:

```
+ - + - + -
```

Perfect wiggle sequence.

13. What happens with equal elements?

Consider:

```
[1, 1, 1, 1]
```

Differences:

```
0, 0, 0
```

Zero isn't positive or negative.

Therefore we cannot extend the wiggle sequence.

```
up = 1
down = 1
```

Answer:

```
1
```

Another example:

```
[1, 7, 7, 4]
```

We get:

```
1 → 7 = UP
7 → 7 = ZERO
7 → 4 = DOWN
```

The equal value doesn't help.

Answer:

3

For example:

1 → 7 → 4

14. Example 3

[1,2,3,4,5,6,7,8,9]

Every movement is UP:

+ + + + + + +

There is no direction change.

So:

```
up = 2
down = 1
```

Answer:

2

We can choose:

[1,2]

or any two different elements.

15. Why this is O(n)

We scan the array exactly once:

```
C++
for (int i = 1; i < n; i++)
```

Each iteration does only constant work.

Therefore:

Time

O(n)

Space

We only use:

```
up
down
```

So:

O(1)

Final intuition to remember

When you see **Wiggle Subsequence**, think:

```
We only care about direction:
UP → DOWN → UP → DOWN ...
```

Maintain:

```
up   = best length ending with UP
down = best length ending with DOWN
```

Then:

```
nums[i] > nums[i-1]
  ↓
  UP movement
  ↓
up = down + 1
```

and:

```
nums[i] < nums[i-1]
  ↓
  DOWN movement
  ↓
down = up + 1
```

Equal:

```
nums[i] == nums[i-1]
  ↓
  ignore
```

So the whole solution is essentially:

```
C++
int up = 1, down = 1;

for (int i = 1; i < nums.size(); i++) {
    if (nums[i] > nums[i - 1])
        up = down + 1;
    else if (nums[i] < nums[i - 1])
        down = up + 1;
}

return max(up, down);
```

Core pattern: this is a classic example where a seemingly $O(n^2)$ DP can be compressed into **two states**, giving the required **$O(n)$ time and $O(1)$ space**.

ChatGPT can make mistakes. Check important info.